

PART 1

Node.js File System (fs) Research Report

readFile() | writeFile() | appendFile() | unlink()

Project Objective

Research four core Node.js fs methods by explaining what each method does, its syntax, practical usage, and how errors are handled. The examples use the modern Promise-based API because it is clear, readable, and easy to scale.

Written by : Karim Khaled Ismail Ahmed

Under the supervision of : Huma Holvo

Mentor : Eng. Mahmoud Montasser

Overview

Node.js provides the built-in fs module for working with files and directories. It supports several programming styles, including callbacks, synchronous methods, and promises. In this report, the Promise API from `node:fs/promises` is used in most examples because it works naturally with `async/await` and keeps error handling explicit.

Core idea

These four operations cover the basic file lifecycle: read existing content, create or replace content, add content without deleting what is already there, and delete a file.

At a glance

Method	Purpose	Typical effect	Common error
<code>fs.readFile()</code>	Read file contents	Gets data without modifying the file	ENOENT, EACCES
<code>fs.writeFile()</code>	Create or overwrite a file	Replaces existing content by default	EACCES, ENOSPC
<code>fs.appendFile()</code>	Add data to the end	Preserves old content and appends new content	ENOENT, EACCES
<code>fs.unlink()</code>	Delete a file	Removes the directory entry for a file	ENOENT, EPERM/EACCES

1. fs.readFile()

Read data from a file without changing it

`fs.readFile()` asynchronously reads the entire contents of a file. The returned data can be a Buffer by default or a string when an encoding such as utf8 is supplied. It is useful when an application needs the complete contents of a relatively small file at once.

Syntax

```
fs.readFile(path[, options])
```

Example

```
const fs = require('node:fs/promises');

async function readConfig() {
  const data = await fs.readFile('config.json', 'utf8');
  console.log(data);
}

readConfig().catch(console.error);
```

Parameters

Parameter	Meaning
path	Path to the file. Can be a string, Buffer, URL, or file descriptor.
options	Optional encoding and flag settings. Using utf8 returns a string; otherwise the result is normally a Buffer.

Error handling

What happens on error?

If the file does not exist, the promise rejects with an error commonly containing code ENOENT. Permission problems can produce EACCES. The error should be handled with `try/catch` around `await` or with `.catch()` on the promise.

Practical note: reading the whole file into memory is convenient, but very large files may be better handled with streams so the application does not load everything at once.

2. fs.writeFile()

Create a new file or replace the existing file content

fs.writeFile() writes data to a file. By default, it creates the file when it does not exist and replaces the existing file contents when it does. It can write text, Buffers, or other supported data types. This makes it a common choice for generating reports, saving JSON, or replacing configuration content.

Syntax

```
fs.writeFile(file, data[, options])
```

Example

```
const fs = require('node:fs/promises');

async function saveMessage() {
  await fs.writeFile('message.txt', 'Hello from Node.js!', 'utf8');
  console.log('File saved.');
}

saveMessage().catch(console.error);
```

Parameters

Parameter	Meaning
file	Destination file path, file handle, or supported URL.
data	Text or binary data to write.
options	Optional encoding, flag, and mode settings. The default flag is commonly w, which creates or truncates the file.

Error handling

What happens on error?

Errors can occur when the destination directory is missing, access is denied, the disk is full, or the path is invalid. A robust application catches the rejection and decides whether to retry, report the problem, or use a fallback location.

Important behavior: writeFile() is not the same as appendFile(). Unless the flags are changed, existing file contents are truncated before the new data is written.

3. fs.appendFile()

Add new data to the end of a file

fs.appendFile() adds data after the current end of a file. Unlike writeFile(), it does not replace the existing content. It is useful for simple logs, history files, or accumulating output over time. If the file does not exist, appendFile() creates it.

Syntax

```
fs.appendFile(path, data[, options])
```

Example

```
const fs = require('node:fs/promises');

async function logEvent(event) {
  await fs.appendFile('app.log', `${new Date().toISOString()} ${event}\n`, 'utf8');
}

logEvent('Server started').catch(console.error);
```

Parameters

Parameter	Meaning
path	File path or supported file identifier.
data	Text or binary data to append.
options	Optional encoding, mode, and flag settings.

Error handling

What happens on error?

If the path is not usable or access is denied, the promise rejects. A common missing-file scenario is handled automatically because appendFile() normally creates the file when it does not already exist; however, a missing parent directory still causes an error such as ENOENT.

Practical note: appendFile() is convenient for simple logging, but high-volume or multi-process logging may require a dedicated logging system or a stream-based design.

4. fs.unlink()

Delete a file

`fs.unlink()` removes a file from the file system. It is the Promise-based equivalent of the traditional asynchronous unlink operation. It should be used for files; directories require different APIs such as `rmdir()` or `rm()` depending on the situation.

Syntax

```
fs.unlink(path)
```

Example

```
const fs = require('node:fs/promises');

async function removeTempFile() {
  await fs.unlink('temp.txt');
  console.log('Temporary file deleted.');
}

removeTempFile().catch(console.error);
```

Parameters

Parameter	Meaning
path	Path to the file that should be removed.

Error handling

What happens on error?

If the file does not exist, the promise rejects with `ENOENT`. Permission restrictions can produce `EACCES` or `EPERM`. Because deletion is destructive, applications often check that the path is the intended target and handle failure explicitly instead of ignoring the error.

Practical note: `unlink()` removes the file name from the file system. It does not provide a recovery mechanism such as a recycle bin, so accidental deletion should be prevented by validating paths carefully.

Common Error Handling Pattern

For the Promise API, the most readable pattern is `async/await` inside `try/catch`. The error object includes useful properties such as `code` and `message`, allowing the program to respond differently to a missing file, permission problem, or storage issue.

```
const fs = require('node:fs/promises');

async function example() {
  try {
    const text = await fs.readFile('data.txt', 'utf8');
    console.log(text);
  } catch (error) {
    console.error('Operation failed:', error.code, error.message);
  }
}

example();
```

Typical fs error codes

Code	Meaning	Example situation
ENOENT	No such file or directory	Reading a missing file or deleting a path that does not exist
EACCES	Permission denied	The process cannot read or write the target
EPERM	Operation not permitted	The operating system blocks the requested operation
ENOSPC	No space left on device	A write or append cannot complete because storage is full

Comparison and Selection Guide

Choose the method based on the intended lifecycle operation rather than the file extension. JSON, TXT, CSV, logs, and other ordinary files can all use these APIs.

Goal	Method	Changes file?	Creates missing file?	Key point
Read existing data	<code>readFile()</code>	No	No	Returns the complete contents
Create/replace data	<code>writeFile()</code>	Yes	Yes	Overwrites by default
Add data	<code>appendFile()</code>	Yes	Yes	Keeps existing contents
Delete a file	<code>unlink()</code>	Yes	No	Removes the file

Mini workflow example

```
const fs = require('node:fs/promises');

async function workflow() {
  try {
```

```
await fs.writeFile('notes.txt', 'First line\n', 'utf8');
await fs.appendFile('notes.txt', 'Second line\n', 'utf8');

const result = await fs.readFile('notes.txt', 'utf8');
console.log(result);

await fs.unlink('notes.txt');
} catch (error) {
  console.error(error.code, error.message);
}
}

workflow();
```

Key Takeaways

- `readFile()` is used to retrieve file content, while `writeFile()` creates or replaces it.
- `appendFile()` adds content to the end without removing the existing content.
- `unlink()` deletes a file and should be used carefully because the operation is destructive.
- All four methods can reject/return errors; production code should handle failures explicitly with `try/catch` or promise catch handlers.
- For very large files or continuous data, streams can be more appropriate than loading or writing everything in one operation.

Final takeaway

Think of the four methods as four basic verbs: READ, WRITE, APPEND, DELETE. Once the intended verb is clear, selecting the correct fs method becomes straightforward.

Technical Note

The examples in this report use the built-in `node:fs/promises` module and `async/await`. Equivalent callback-based methods exist in `node:fs`, including `fs.readFile()`, `fs.writeFile()`, `fs.appendFile()`, and `fs.unlink()`. The underlying operations and common operating-system error codes are the same concepts presented here.